Light Distillation for Calibration-Permitted Low-Rank Attention Compression

An optional companion to GRC

HyperTensor Research

April 2026

Abstract

Paper A introduced GRC, a calibration-free shared-basis low-rank projection of attention weights that is fast on consumer GPUs (+6.27% throughput at k=1024 on Llama-3.1-8B Q4_K_M, RTX 4070 Laptop) but pays a measured +13.30% perplexity penalty at the quality-preserving rank k=1536. This companion paper introduces an optional, opt-in distillation step that recovers a configurable fraction of that penalty in exchange for a few hundred forward passes on a small calibration corpus. The contribution is a protocol, not new math: per-layer rank-r LoRA adapters [1] fit on the teacher-student logit residual after GRC projection, with sink-channel exemption [2, 3] as an orthogonal quality lever. We describe the design, the runtime path that allows distillation to remain optional (the calibration-free GRC path stays the default and the distilled adapters are mergeable), and the empirical protocol that an EC2 GPU run should follow. *Status: this paper is a scaffold. Empirical results, baselines and ablations will be filled in once the distillation runner has executed on appropriate hardware. Section 3 tracks the gating items.*

Contents

1 Motivation **3**

2 Method **3**

2.1 Phase 1: GRC projection (unchanged from Paper A) 3

2.2 Phase 2: Teacher-student LoRA correction (new) 3

2.3 Phase 3: Merge and ship 3

3 Implementation status **3**

4 Why distillation does not break the fusion path **4**

5 Reproducibility **4**

1 Motivation

Paper A is deliberately calibration-free: GRC requires no data, no fine-tuning, no per-deployment configuration beyond a single rank knob. That property is valuable for the deployment surface it targets (consumer GPUs, ad-hoc compression, no MLOps stack), and the empirical trade is that there is a quality floor set by the joint Gram spectrum itself. On Llama-3.1-8B-Instruct the floor at k=1024 is roughly the bottom 9% of joint Q+K+V energy (k_{95}≈1682), and the realised perplexity penalty is +13.30% at k=1536 on WikiText-2 [**merity2017wikitext**]. Paper A's Limitations section decomposes that penalty into intrinsic floor and reducible implementation choice.

This paper exercises the largest reducible lever, light distillation, behind an opt-in flag. The design constraint is that turning it on must not break Paper A's runtime path; turning it off must reproduce Paper A's numbers exactly.

2 Method

2.1 Phase 1: GRC projection (unchanged from Paper A)

For each attention block, build the shared basis $\mathbf{P}_t$ from the joint Gram of $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ and form the projected weights $\mathbf{W}'_X = \mathbf{W}_X \mathbf{P}_k \mathbf{P}_k^\top$. Optionally, exempt the top-T highest-L2 columns from projection (sink-aware GRC, see Paper A Table 6 / `sink_channel_pilot.json`). This phase is pure NumPy and runs on any host.

2.2 Phase 2: Teacher-student LoRA correction (new)

Fit per-layer rank-r LoRA adapters $_X \in \mathbb{R}^{m\times r}$, $_X \in \mathbb{R}^{r\times d}$ for $X \in \{Q, K, V\}$ such that

$$\hat{\mathbf{W}}_X = \mathbf{W}'_X +_X {}_X$$

minimises a teacher-student logit MSE over a small calibration corpus (target: 5,000 to 10,000 tokens, sequence length 512, batch 8; 500-step schedule on AdamW with η=10^{-4}, weight decay 0, cosine LR with warmup-100). The teacher is the original uncompressed model; the student is the projected model with frozen $\mathbf{W}'_X$ and trainable $(_X, _X)$. Only the attention QKV adapters are fit; output projection $\mathbf{W}_O$ and FFN weights remain at baseline. This holds the parameter budget below $3rdL = 3 \cdot 8 \cdot 4096 \cdot 32 \approx 3.1$ M trainable weights for L=32 layers and r=8, two orders of magnitude below full LoRA fine-tuning.

2.3 Phase 3: Merge and ship

The trained LoRA factors are folded into the projected weights ($\hat{\mathbf{W}}_X = \mathbf{W}'_X +_X {}_X$) and re-quantised to Q4_K_M via the standard llama.cpp quantize path. The runtime loads the resulting GGUF unchanged; no runtime code changes are needed. If the user disables distillation, Paper A's pipeline runs unmodified.

3 Implementation status

- **Done.** Phase 1 reference implementation in `scripts/analysis/sink_channel_grc.py` and `scripts/grc_distill.py` (NumPy; CPU-only).

- **Done.** Sink-channel pilot (Paper A Table 6) showing calibration-free sink exemption alone yields 1 to 3% relative reconstruction-error improvement at $k \in \{512, 1024, 1536\}$.
- **Pending.** Phase 2 PyTorch runner. Requires a GPU host with at least one A100 / L40S / H100 to hold the teacher in BF16 with gradient activations. EC2 `g5.xlarge` or `g6.xlarge` is sufficient for an 8B model.
- **Pending.** WikiText-2 PPL evaluation on the merged artifact. The expected operating regimes:
  1. Vanilla GRC at k=1024: +13.30% PPL (Paper A baseline).
  2. Sink-aware GRC at k=1024, T=32: predicted +12.5% to +13.0% (small effect; the calibration-free magnitude proxy weakly correlates with runtime sink channels).
  3. Distilled GRC at k=1024, r=8, 5,000-token corpus: target +5% to +8% PPL (50 to 60% gap closure).
  4. Distilled sink-aware GRC at k=1024, T=32, r=8: target +3% to +5%.

  These targets are predictions, not measurements. The paper will be updated with measured numbers once the runner has executed.
- **Pending.** Throughput re-measurement on the distilled artifact. Hypothesis: throughput is unchanged from vanilla GRC, because the merged adapters do not alter the kernel-fusion path (§ 4). If measurement contradicts the hypothesis, that becomes a result.

4 Why distillation does not break the fusion path

Paper A established that the GRC throughput improvement comes from kernel fusion: three `gemv_q4_k` calls collapse into a fused `gemv_dual_q8_0` trio operating on k-dimensional intermediates. The fusion-viability condition is that the projected weights $\mathbf{W}'_X$ have rank k. The merged weights $\hat{\mathbf{W}}_X = \mathbf{W}'_X +_{X\,X}$ in general have rank $k + r$ (when $r \leq m - k$), but the $_{XX}$ term has rank exactly r and is small; the runtime can either (a) treat $\hat{\mathbf{W}}_X$ as a full-rank matrix and lose fusion (the +6.27% throughput gain reverts), (b) keep $\mathbf{W}'_X$ on the fused path and run the LoRA correction as a separate small matmul-add (cost $\sim r/k$ of the main GEMV; negligible at r=8, k=1024), or (c) re-quantise the merged $\hat{\mathbf{W}}$ to Q4_K_M at rank k via a second SVD pass and accept the merge-loss. Default recommendation is (b); (c) is reserved for the lowest-VRAM target where adapter storage matters.

5 Reproducibility

The reference Phase-1 code lives in `scripts/analysis/sink_channel_grc.py` and `scripts/grc_distill.py`. The Phase-2 runner will live in `scripts/grc_distill_runner.py` (PyTorch + transformers) and the EC2 driver in `scripts/ec2_paperE_distill/`. A complete empirical reproduction will be published alongside the populated results once the runner has executed.

References

[1] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: 2106.09685 [cs.CL].

[2] M. Sun, X. Chen, J. Z. Kolter, and Z. Liu. *Massive Activations in Large Language Models*. 2024. arXiv: 2402.17762 [cs.CL].

[3] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. *Efficient Streaming Language Models with Attention Sinks*. 2024. arXiv: 2309.17453 [cs.CL].